↳ Every modern processor is different & complex containing many registers, Multiple arithmetic unit.

↳ We will focus on a simple processor called 'the basic computer'

Basic Computer

↳ It has two components:

1. Processor
2. Memory → has 4096 words with 12 bit address width & 16 bit memory width.

Instruction:

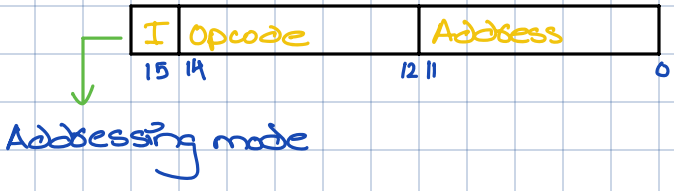
↳ A sequence of machine instructions is called program.

↳ A group of bits that tell the computer to perform a specific operation (a sequence of micro-operations) are called machine instructions.

↳ These instructions along with data is stored in memory.

↳ It is placed in Instruction Register (IR) for processing (Translated by Control unit into a sequence of micro-operations necessary to implement it).

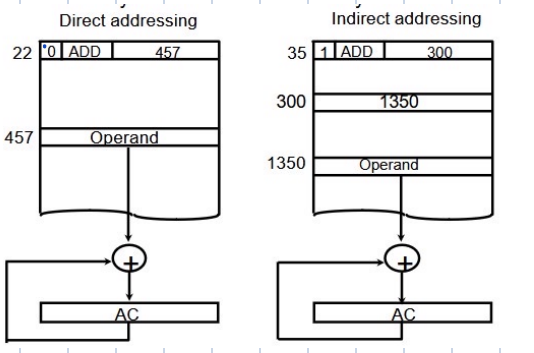
↳ IR has following format:



Addressing Modes:

↳ There are two Modes:

1. Direct address (Effective address)
   ↳ Address directly leads to desired value.
2. Indirect address:
   ↳ Address leads to another address which leads to the desired value.

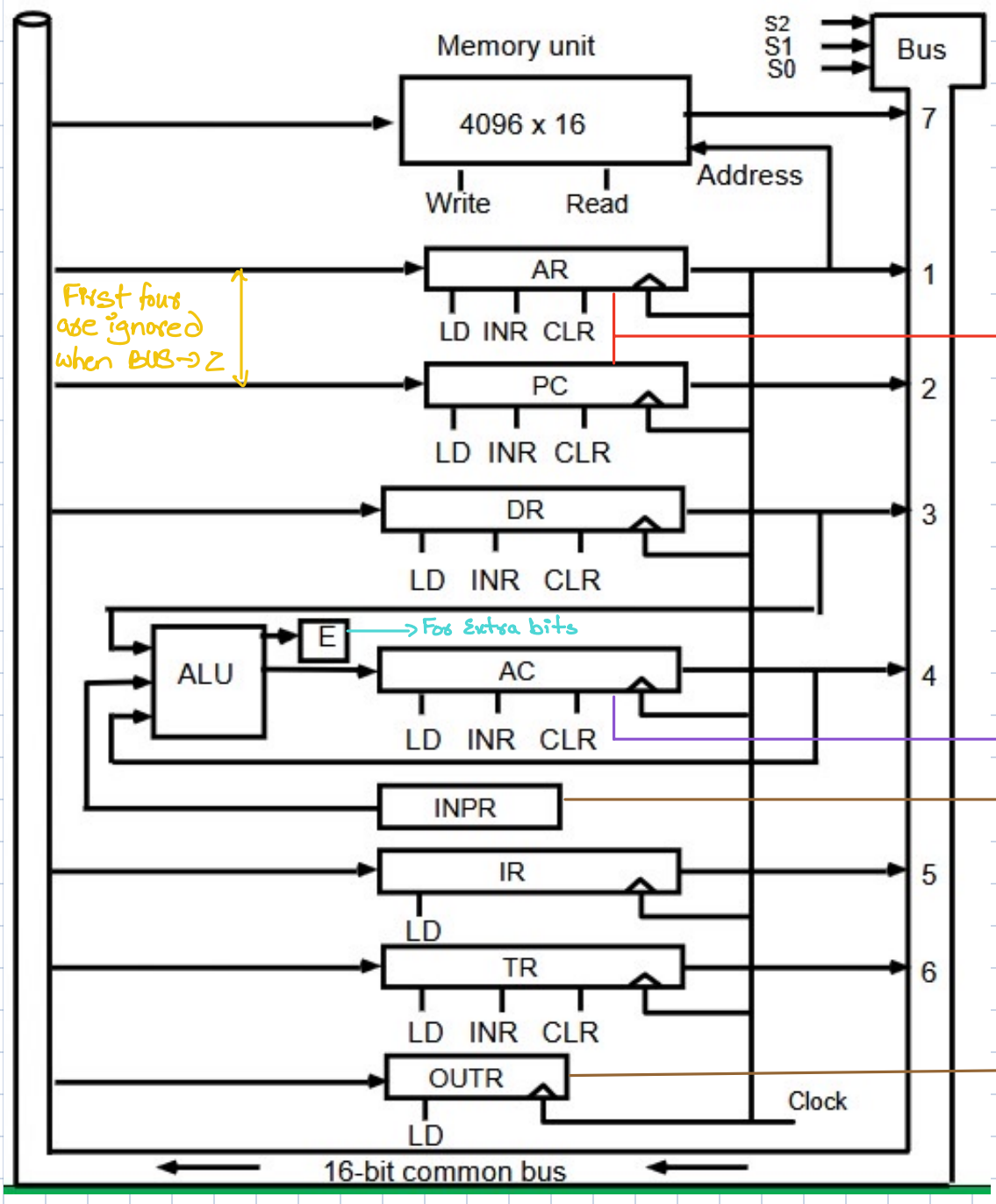


Processor Registers

DR	16	Data Register	Holds memory operand
AR	12	Address Register	Holds address for memory
AC	16	Accumulator	Processor register
IR	16	Instruction Register	Holds instruction code
PC	12	Program Counter	Holds address of next instruction
TR	16	Temporary Register	Holds temporary data
INPR	8	Input Register	Holds input character
OUTR	8	Output Register	Holds output character

General purpose register

Common Bus System:



LD = Load
INR = increment
CLR = clear

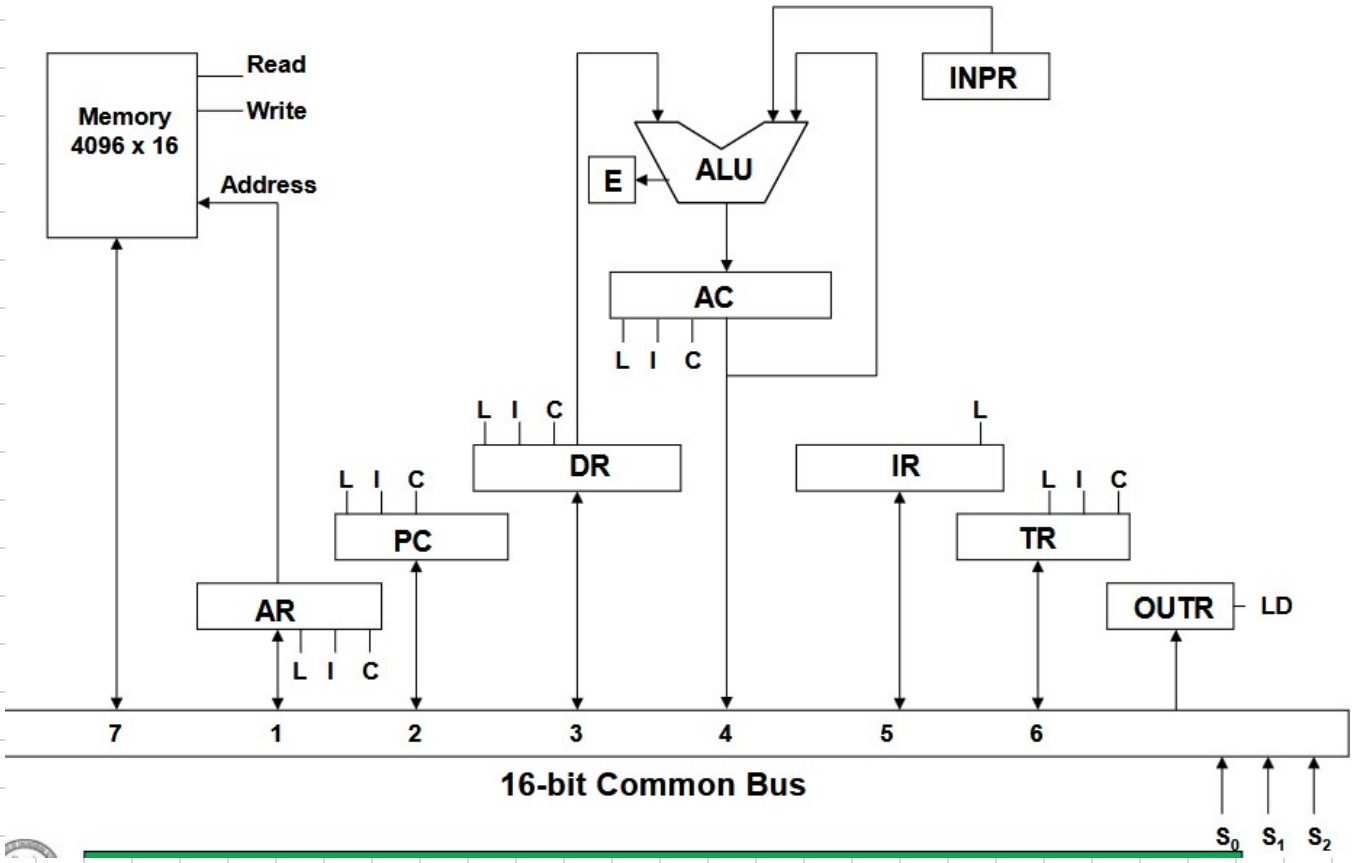
Z = AR/PC

Z(12-15) = 0
Z(0-11) = address

AC has no direct BUS → AC. It needs to go through ALU.

Can hold only 8 bit character. From BUS first 8 bits are ignored.

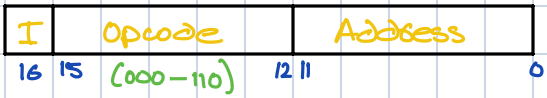
A simplified view:



S_2	S_1	S_0	Register
0	0	0	X
0	0	1	AR
0	1	0	PC
0	1	0	DR
1	0	1	AC
1	0	1	IR
1	1	0	TR
1	1	1	Memory

Basic computer instructions:

1. Memory Reference instruction:



2. Register-Reference Instructions

3. Input-Output instructions

For example,

Symbol	Hex Code I = 0	Hex Code I = 1	Description
AND	0xxx	8xxx	AND memory word to AC
ADD	1xxx	9xxx	Add memory word to AC
LDA	2xxx	Axxx	Load AC from memory
STA	3xxx	Bxxx	Store content of AC into memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear AC
CLE	7400		Clear E
CMA	7200		Complement AC
CME	7100		Complement E
CIR	7080		Circulate right AC and E
CIL	7040		Circulate left AC and E
INC	7020		Increment AC
SPA	7010		Skip next instr. if AC is positive
SNA	7008		Skip next instr. if AC is negative
SZA	7004		Skip next instr. if AC is zero
SZE	7002		Skip next instr. if E is zero
HLT	7001		Halt computer
INP	F800		Input character to AC
OUT	F400		Output character from AC
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off

↳ A set of instructions that can be used to perform any operation known is reffered to as completeness of instruction set.

↳ Instruction Types:

Functional, Transfer, Control, and I/O instructions.

Control Unit:

↳ A processor that translates machine instructions into control signals for micro-operations that implement them.

↳ Implemented as:

Hardwired

↳ CU is made up of sequential and combinational circuits to generate control signals.

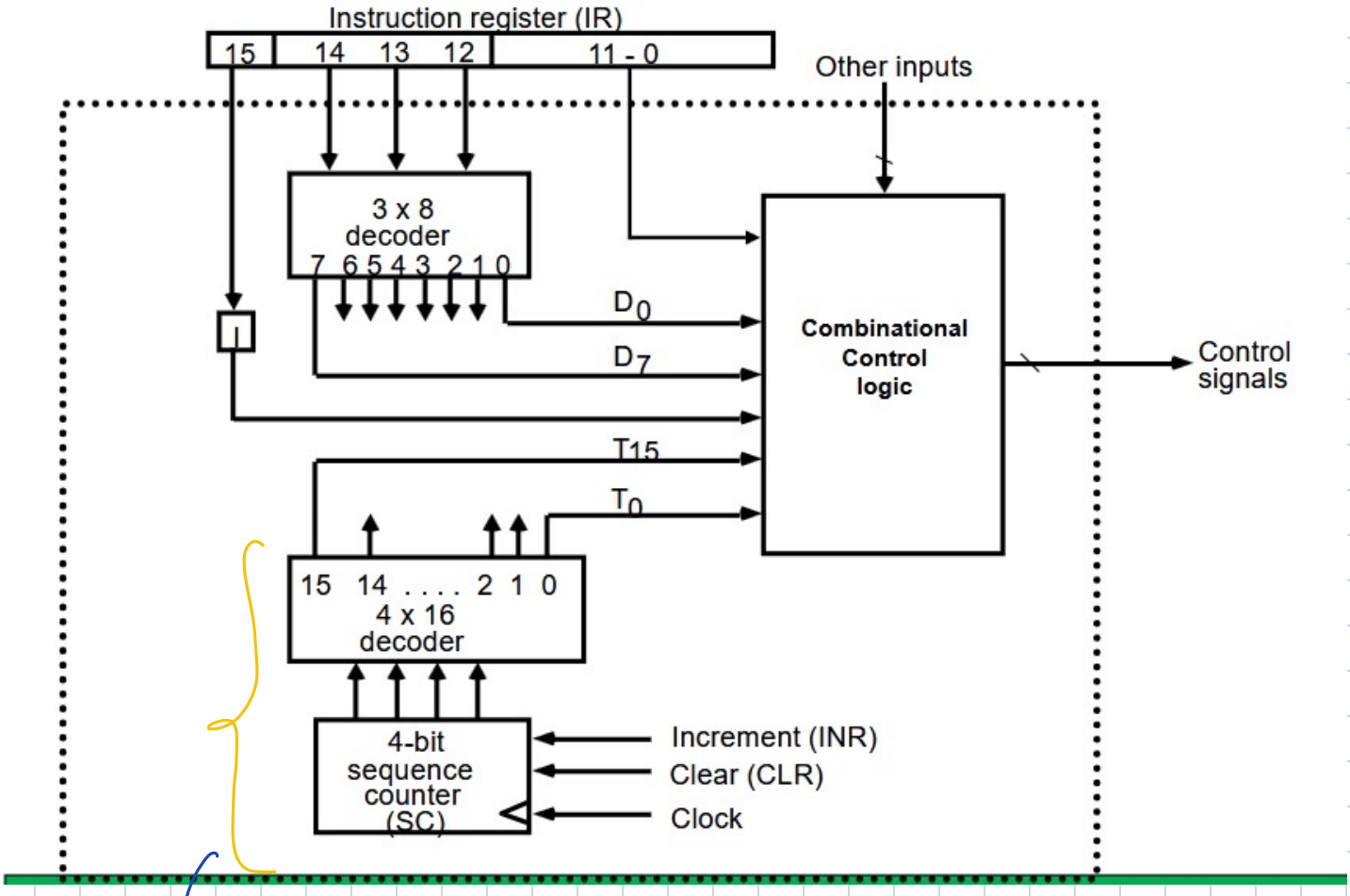
↳ Difficult to repurpose

Microprogrammed

↳ A control memory on the processor contains micro-programs that activate the necessary control signals.

↳ Easy to re-purpose

Basic Computer's CU:



↳ Ensures each step happens in order with a SC

With a clock we would need additional logic to ensure a particular micoperation happens when desired.

↳ In Basic computer, an instruction is executed in following cycle:

1. Fetch an instruction from memory
2. Decode the instructions
3. Read effective address from memory if instruction has an indirect address else skip this
4/3. Execute the instruction.

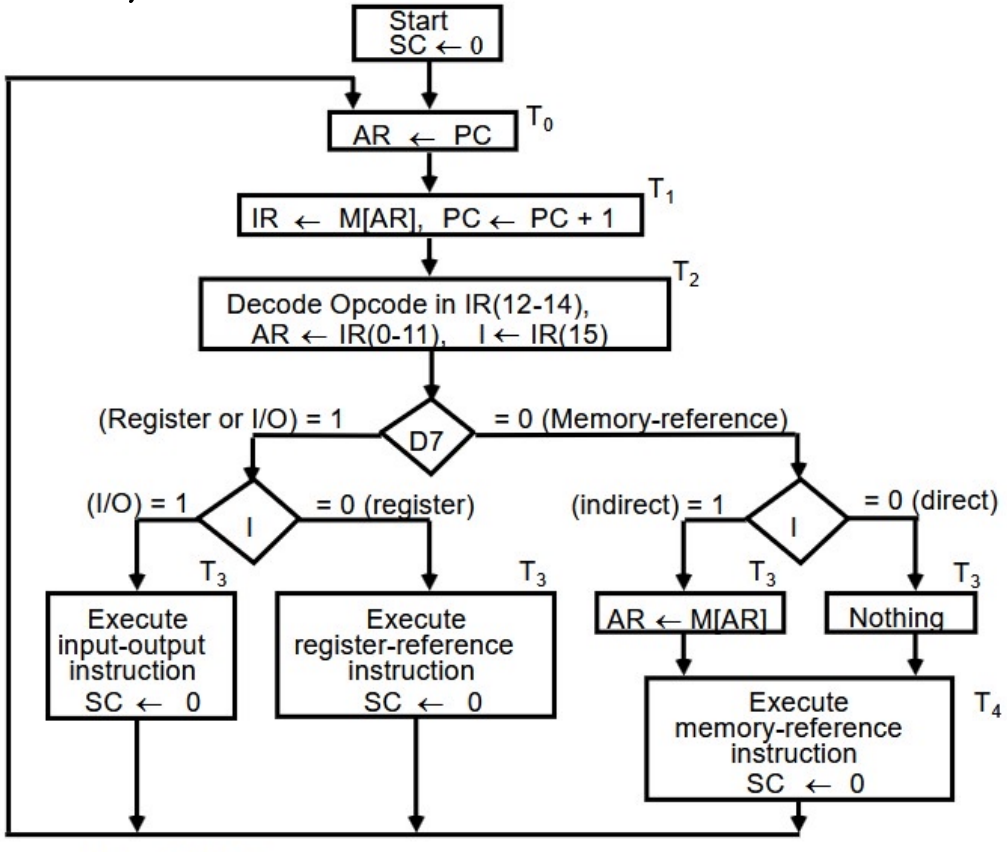
Fetch and Decode:

T_0 : $AR \leftarrow PC$ $(S_0S_1S_2 = 010, T_0 = 1)$

T_1 : $IR \leftarrow M[AR], PC \leftarrow PC + 1$ $(S_0S_1S_2 = 111, T_1 = 1)$

T_2 : $D_0, \ldots, D_7 \leftarrow$ Decode $IR(12-14), AR \leftarrow IR(0-11), I \leftarrow IR(15)$

Determine the type of instruction



$D'7IT_3$: $AR \leftarrow M[AR]$
$D'7I'T_3$: Nothing
$D7I'T_3$: Execute a register-reference instr.
$D7IT_3$: Execute an input-output instr.

Register reference instructions

$D_7 I' T_3 = r$ (common to all register-reference instructions)
$IR(i) = B_i$ [bit in $IR(0\text{–}11)$ that specifies the operation]

All end at T_3 (Advantage)

	r:	$SC \leftarrow 0$	Clear SC
CLA	rB_{11}:	$AC \leftarrow 0$	Clear AC
CLE	rB_{10}:	$E \leftarrow 0$	Clear E
CMA	rB_9:	$AC \leftarrow \overline{AC}$	Complement AC
CME	rB_8:	$E \leftarrow \overline{E}$	Complement E
CIR	rB_7:	$AC \leftarrow \text{shr } AC, AC(15) \leftarrow E, E \leftarrow AC(0)$	Circulate right
CIL	rB_6:	$AC \leftarrow \text{shl } AC, AC(0) \leftarrow E, E \leftarrow AC(15)$	Circulate left
INC	rB_5:	$AC \leftarrow AC + 1$	Increment AC
SPA	rB_4:	If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$	Skip if positive
SNA	rB_3:	If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$	Skip if negative
SZA	rB_2:	If $(AC = 0)$ then $PC \leftarrow PC + 1)$	Skip if AC zero
SZE	rB_1:	If $(E = 0)$ then $(PC \leftarrow PC + 1)$	Skip if E zero
HLT	rB_0:	$S \leftarrow 0$ (S is a start–stop flip-flop)	Halt computer

Memory reference instructions

Symbol	Operation Decoder	Symbolic Description
AND	D_0	**AC ← AC ∧ M[AR]**
ADD	D_1	**AC ← AC + M[AR], E ← C_{out}**
LDA	D_2	**AC ← M[AR]**
STA	D_3	**M[AR] ← AC**
BUN	D_4	**PC ← AR**
BSA	D_5	**M[AR] ← PC, PC ← AR + 1**
ISZ	D_6	**M[AR] ← M[AR] + 1, if M[AR] + 1 = 0 then PC ← PC+1**

- Memory cycle is assumed to be short enough to be completed in CPU cycle. The execution of MR instructions starts from T_4

AND to AC

D_0T_4:	DR ← M[AR]	Read operand
D_0T_5:	AC ← AC ∧ DR, SC ← 0	AND with AC

ADD to AC

D_1T_4:	DR ← M[AR]	Read operand
D_1T_5:	AC ← AC + DR, E ← C_{out}, SC ← 0	Add to AC and store carry in E

LDA: Load to AC

D_2T_4: DR ← M[AR]
D_2T_5: AC ← DR, SC ← 0

} AC is not directly connected to bus

STA: Store AC

D_3T_4: M[AR] ← AC, SC ← 0

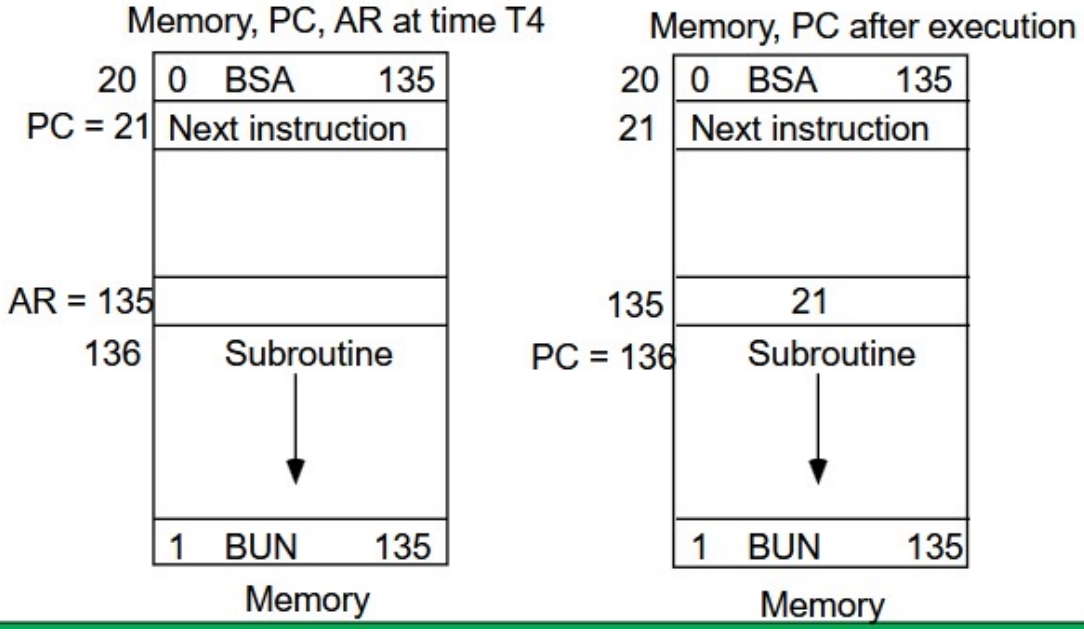
BUN: Branch Unconditionally

D_4T_4: PC ← AR, SC ← 0

Here SC ← SC+1 is implicitly mentioned

BSA: Branch and Save Return Address

M[AR] ← PC, PC ← AR + 1



BSA:

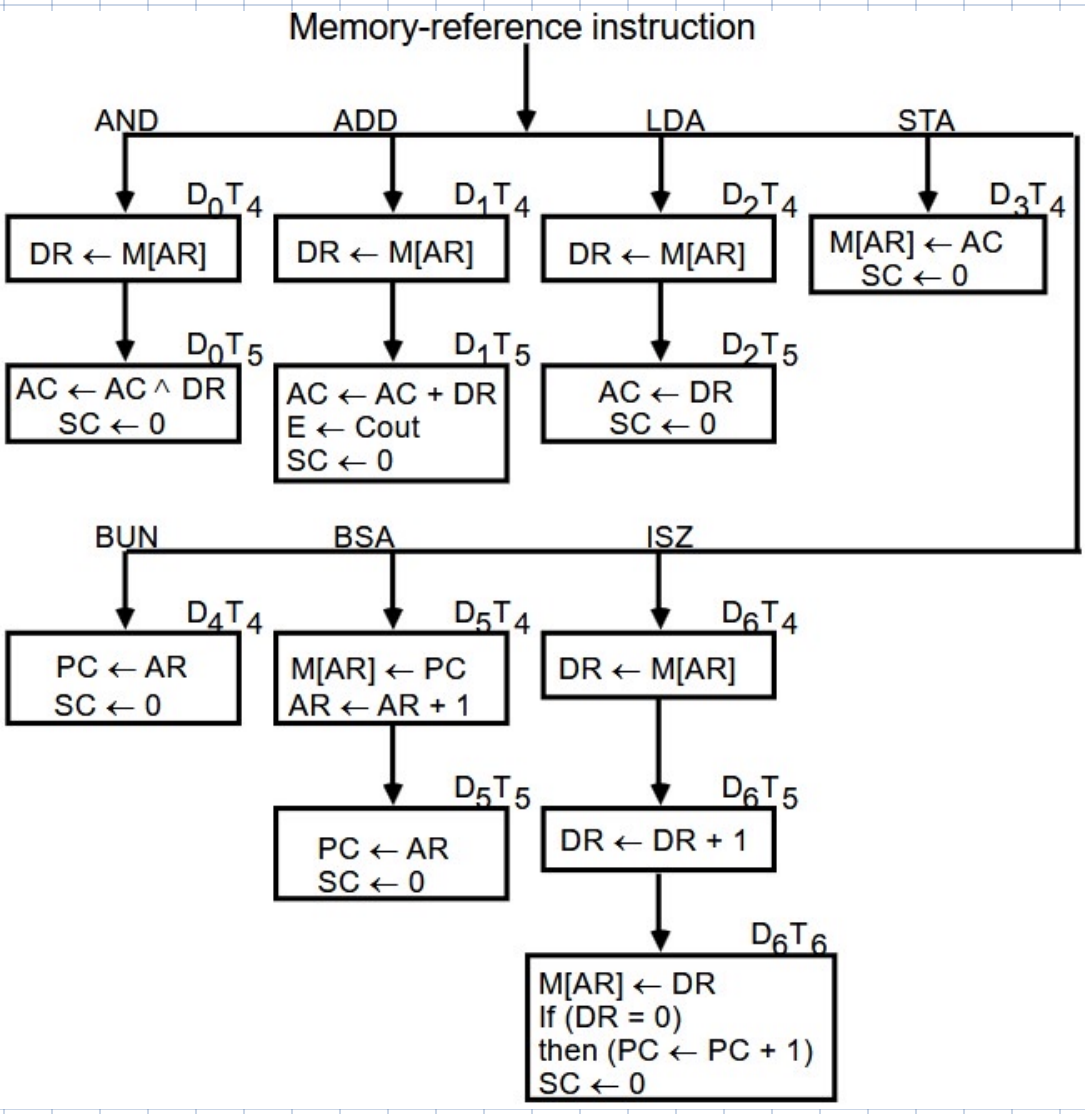
D_5T_4: M[AR] ← PC, AR ← AR + 1
D_5T_5: PC ← AR, SC ← 0

ISZ: Increment and Skip-if-Zero

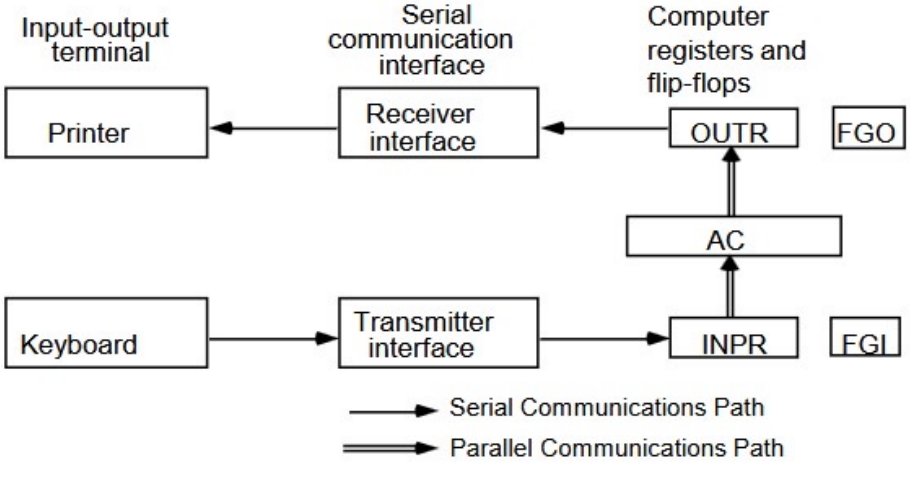
D_6T_4: DR ← M[AR]
D_6T_5: DR ← DR + 1
D_6T_6: M[AR] ← DR, if (DR = 0) then (PC ← PC + 1), SC ← 0

} DR has 2's comp here so that's why on +1, it can be 0.

↳ These instructions happen at same level because they all start from T_4.



Input Output & Interrupt instructions



INPR	Input register - 8 bits
OUTR	Output register - 8 bits
FGI	Input flag - 1 bit
FGO	Output flag - 1 bit
IEN	Interrupt enable - 1 bit

FGI: AC ← INPR
After going to AC will then

FGO: OUTR ← AC

↳ Terminal sends/recieves input/output serially

↳ The flags (FGI & FGO) are needed to synchronize the timing difference b/w I/O device and the computer.

Program controlled Data transfer

CPU:

FGI=0

If FGI=0 GOTO loop
AC ← INPR, FGI ← 0

FGO=1

If FGO=0 GOTO loop
OUTPR ← AC, FGO ← 0

I/O controller:

If FGI=1 GOTO loop
INPR ← new data, FGI ← 1

If FGO=1 GOTO loop
consume OUTR, FGO ← 1

Explanation:

For input:

Initially FGI=0, meaning no new data

↳ IO places data in INPR & sets FGI=1 to tell CPU
"I've got new data for you".

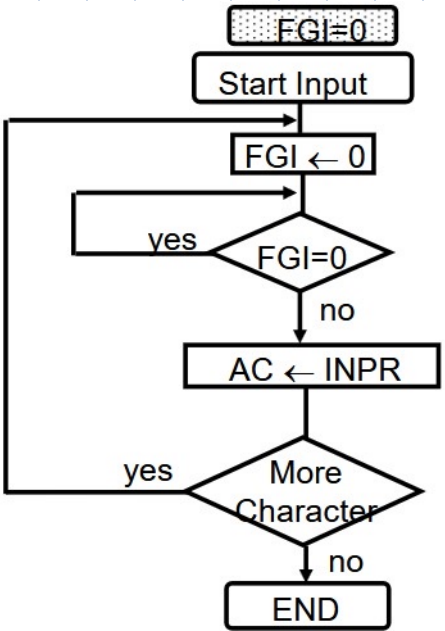
↳ CPU waits till FGI=1 meaning IO sent data.

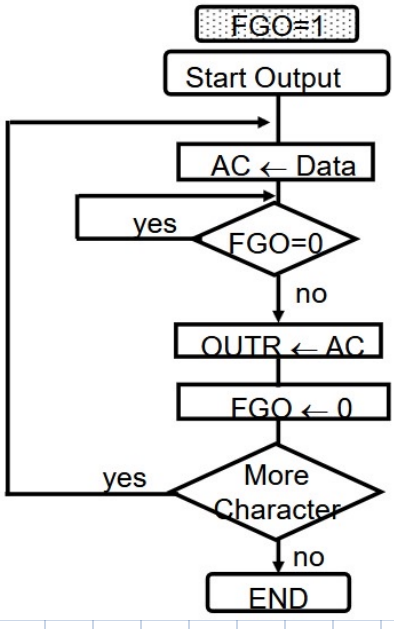
↳ Once FGI is 1, it reads INPR & sets FGI to 0 to tell I/O.
"I've read the data. You can send more".

For output:

↳ The CPU writes data to OUTR & sets FGO=0 to tell IO
"Here data for you".

↳ The IO takes data & processes it. Once done it sets FGO=1
to tell CPU "You can send more now"





$D_7IT_3 = p$
$IR(i) = B_i$, i = 6, …, 11

Ends all at T3 (Advantage)

	p:	SC ← 0	Clear SC
INP	pB_{11}:	AC(0-7) ← INPR, FGI ← 0	Input char. to AC
OUT	pB_{10}:	OUTR ← AC(0-7), FGO ← 0	Output char. from AC
SKI	pB_9:	if(FGI = 1) then (PC ← PC + 1)	Skip on input flag
SKO	pB_8:	if(FGO = 1) then (PC ← PC + 1)	Skip on output flag
ION	pB_7:	IEN ← 1	Interrupt enable on
IOF	pB_6:	IEN ← 0	Interrupt enable off

↳ This is simple & requires least hardware but takes up valuable CPU time & slows CPU to speed of IO.

So we have

Interrupt initiated Input/Output

↳ The IO interface (NOT CPU) continously monitors the IO device

↳ When the IO device is ready for data transfer, the IO interface generates an interrupt request to the CPU.

↳ The CPU, which was doing another task, pauses it's work to handle the interrupt.

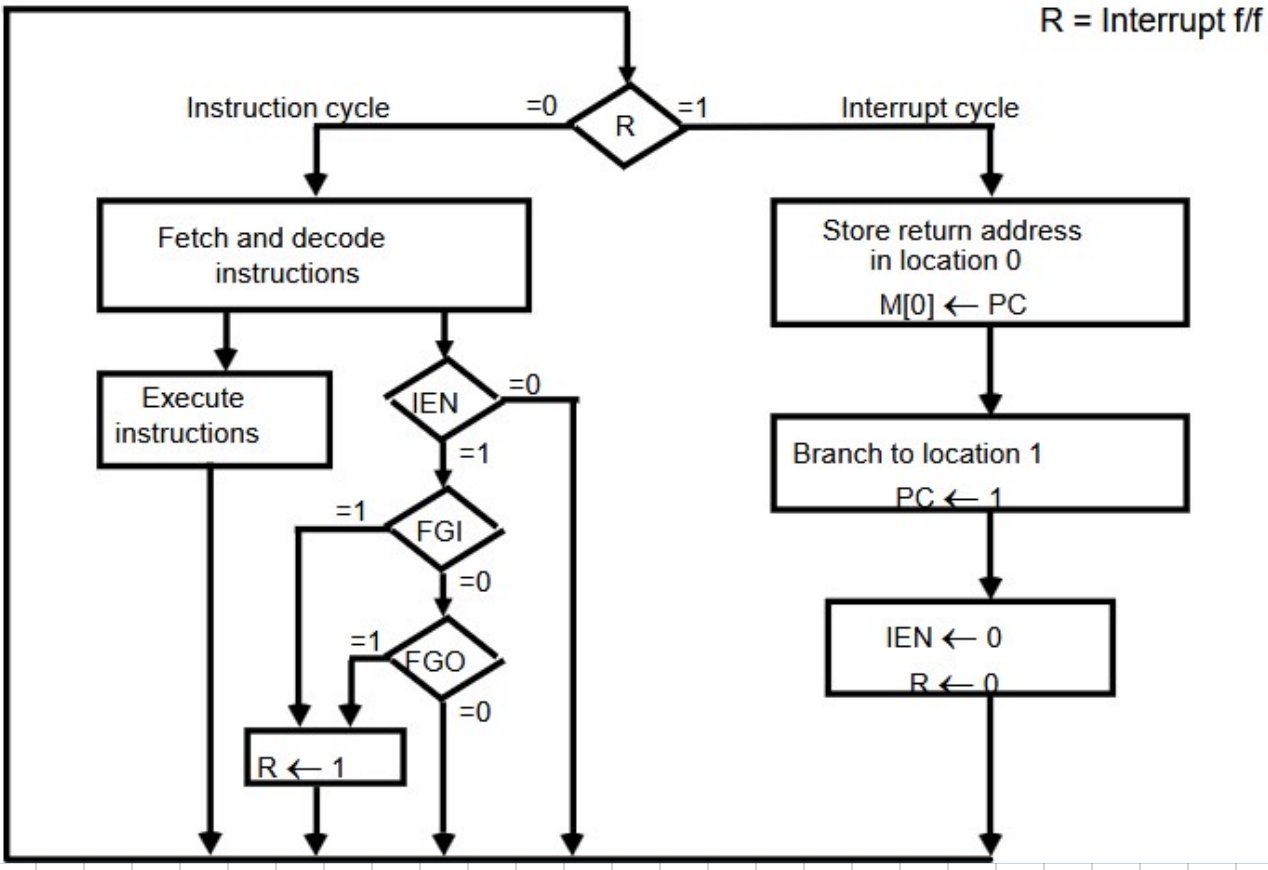
↳ The CPU branches to Interrupt service routine to process IO operation.

↳ After completing the IO operation, the CPU resumes its previous task.

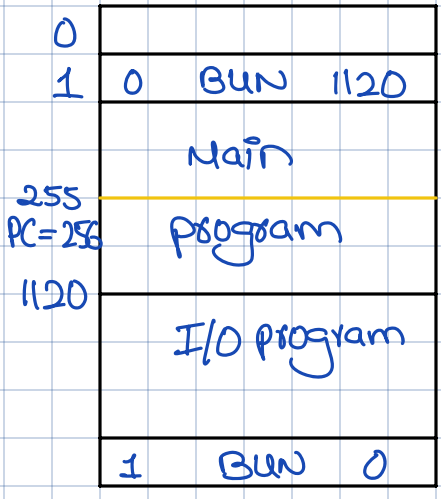
↳ IEN (interrupt-enable flip flop)

↳ Determines whether CPU is interrupted or not.

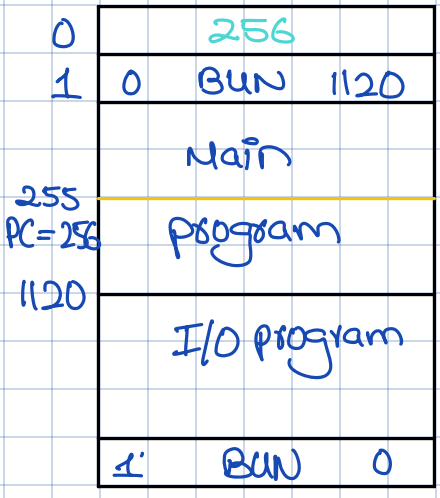
↳ Can be set or cleared by instructions.



Before interrupt



After interrupt



NOTE:
IEN is 0 during interrupt & IEN is 1 other times.

Register transfer operations in interrupt cycle

$R \leftarrow 1 \quad \text{if } IEN(FGI + FGO)T_0'T_1'T_2'$

Now, fetch & decode phases of instruction must be modified to differ from interrupt cycle.

Replace T_0, T_1, T_2 with $\overline{R}T_0, \overline{R}T_1, \overline{R}T_2$

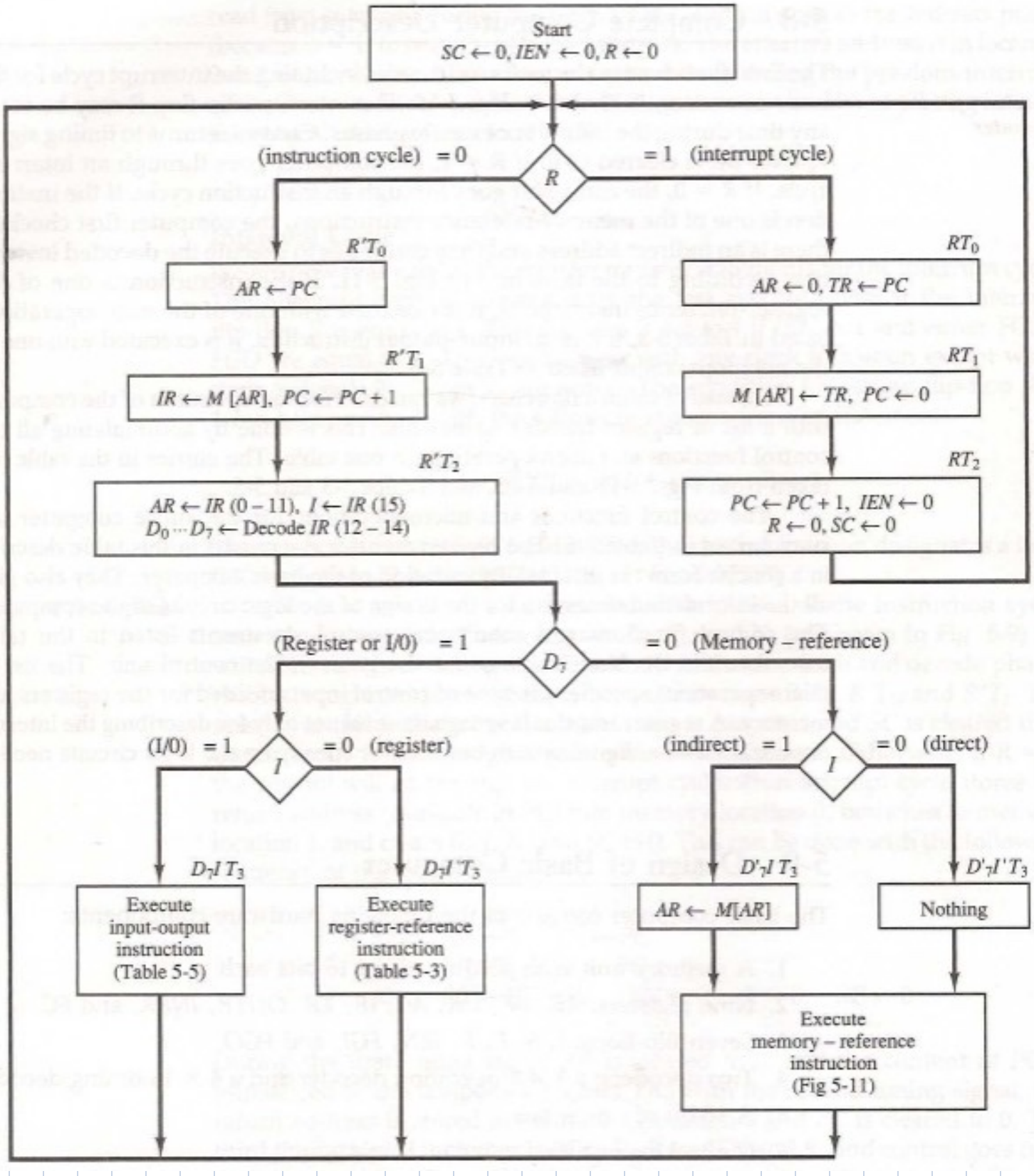
So interrupt cycle is:

RT_0 : $AR \leftarrow 0$, $TR \leftarrow PC$

RT_1 : $M[AR] \leftarrow TR$, $PC \leftarrow 0$

RT_2 : $PC \leftarrow PC+1$, $IEN \leftarrow 0$, $R \leftarrow 0$, $SC \leftarrow 0$

Complete flowchart of operations

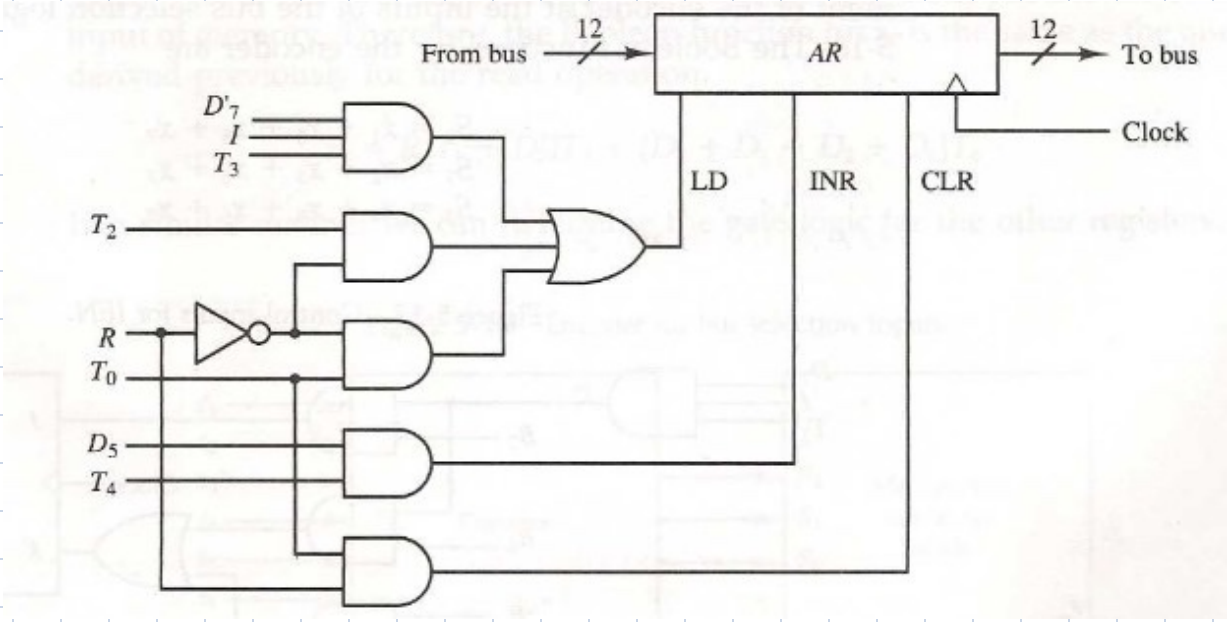


Control of Address Register:

↳ Collecting instructions leads us to below

$R'T_0$:	$AR \leftarrow PC$	$LD(AR)$
$R'T_2$:	$AR \leftarrow IR(0\text{-}11)$	$LD(AR)$
D'_7IT_3:	$AR \leftarrow M[AR]$	$LD(AR)$
RT_0:	$AR \leftarrow 0$	$CLR(AR)$
D_5T_4:	$AR \leftarrow AR + 1$	$INR(AR)$

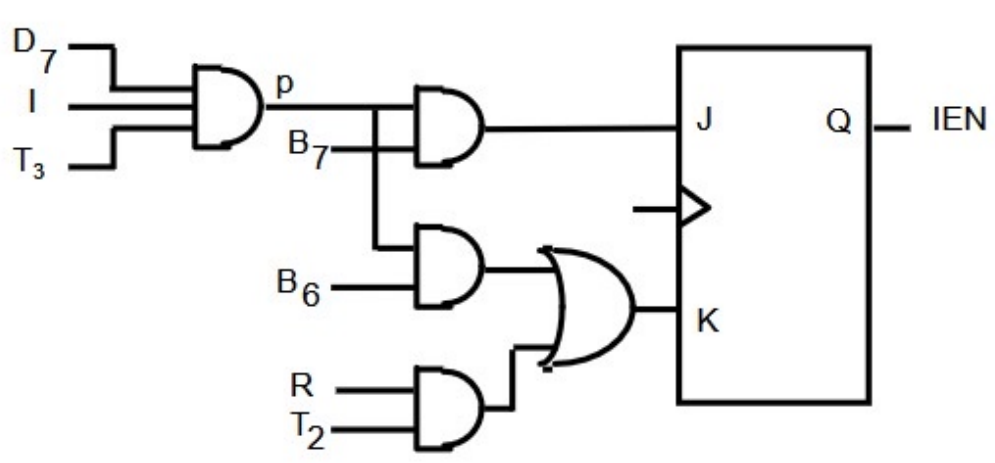
$$LD(AR) = R'T_0 + R'T_2 + D'_7IT_3$$
$$CLR(AR) = RT_0$$
$$INR(AR) = D_5T_4$$



Control of IEN flag

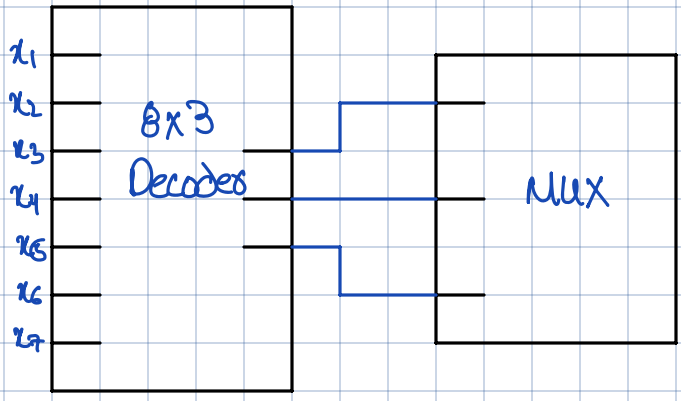
pB_7: $IEN \leftarrow 1$ (I/O Instruction)
pB_6: $IEN \leftarrow 0$ (I/O Instruction)
RT_2: $IEN \leftarrow 0$ (Interrupt)

$p = D_7IT_3$ (Input/Output Instruction)



Control of common bus

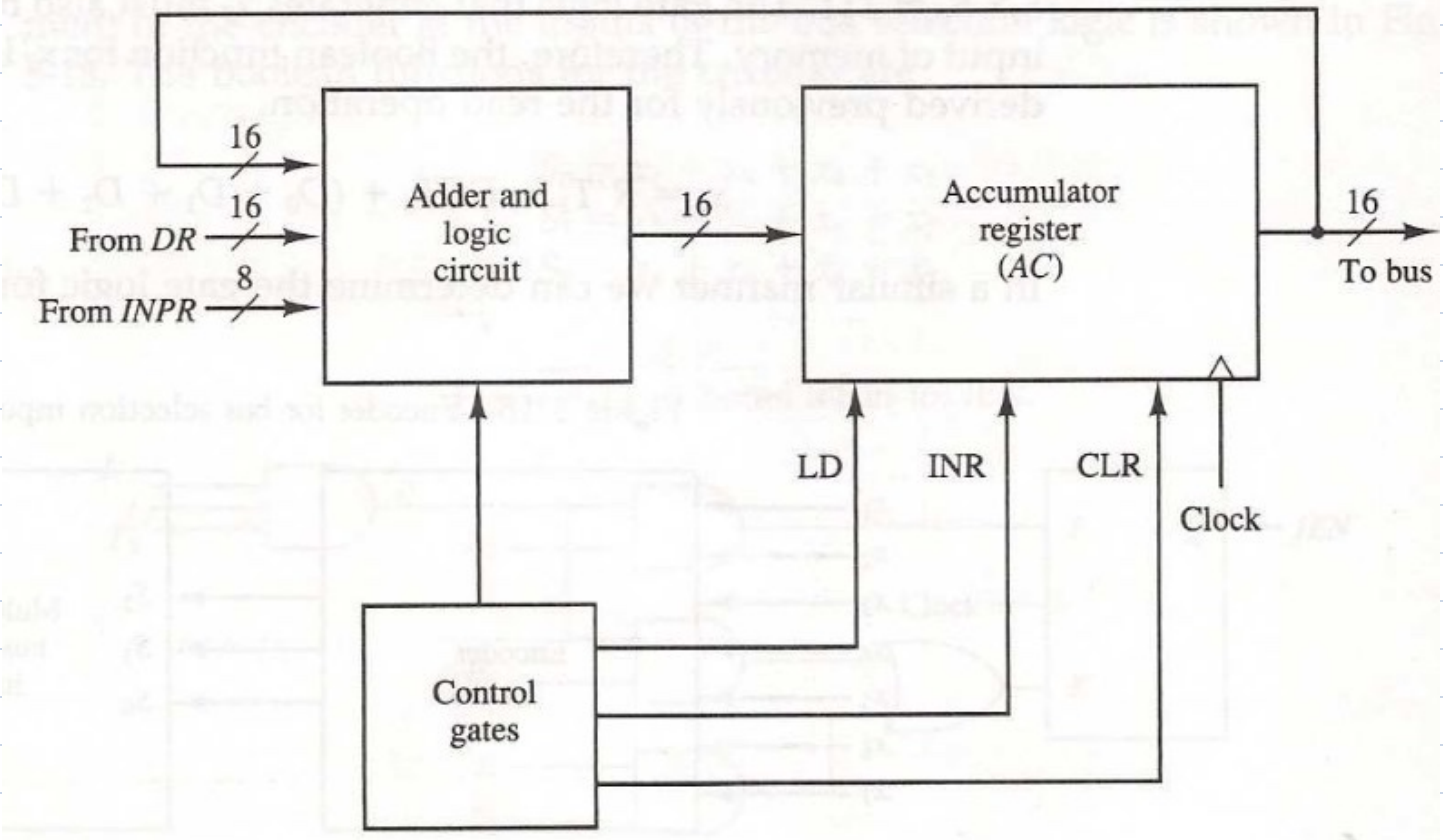
x_1	x_2	x_3	x_4	x_5	x_6	x_7	S_2	S_1	S_0	Selected Register
0	0	0	0	0	0	0	0	0	0	None
1							0	0	1	AR
	1						0	1	0	PC
		1					0	1	1	DR
			1				1	0	0	AC
				1			1	0	1	IR
					1		1	1	0	TR
						1	1	1	1	Memory



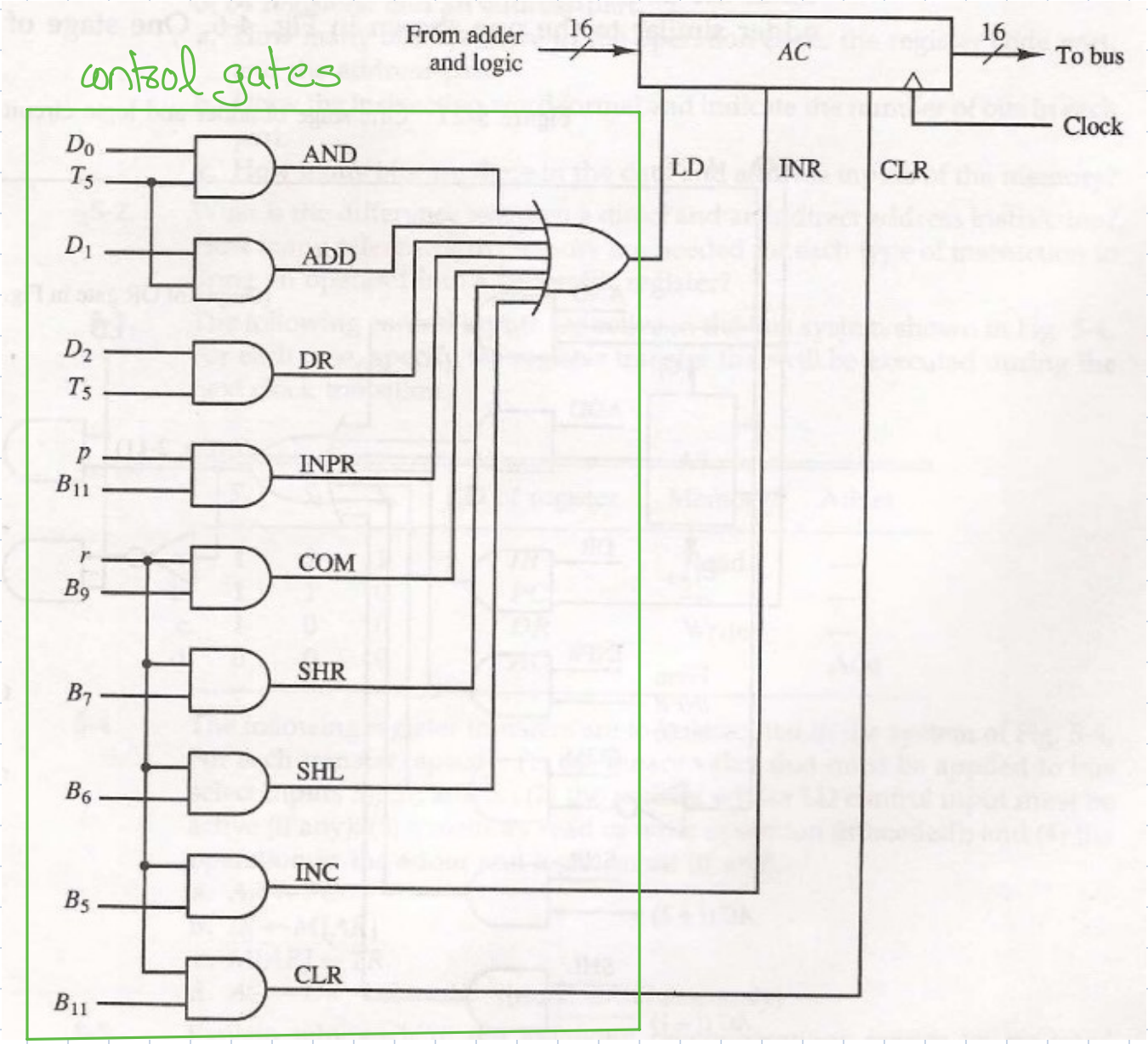
Design of Accumulator Logic:

- All statements that effect the contents of AC

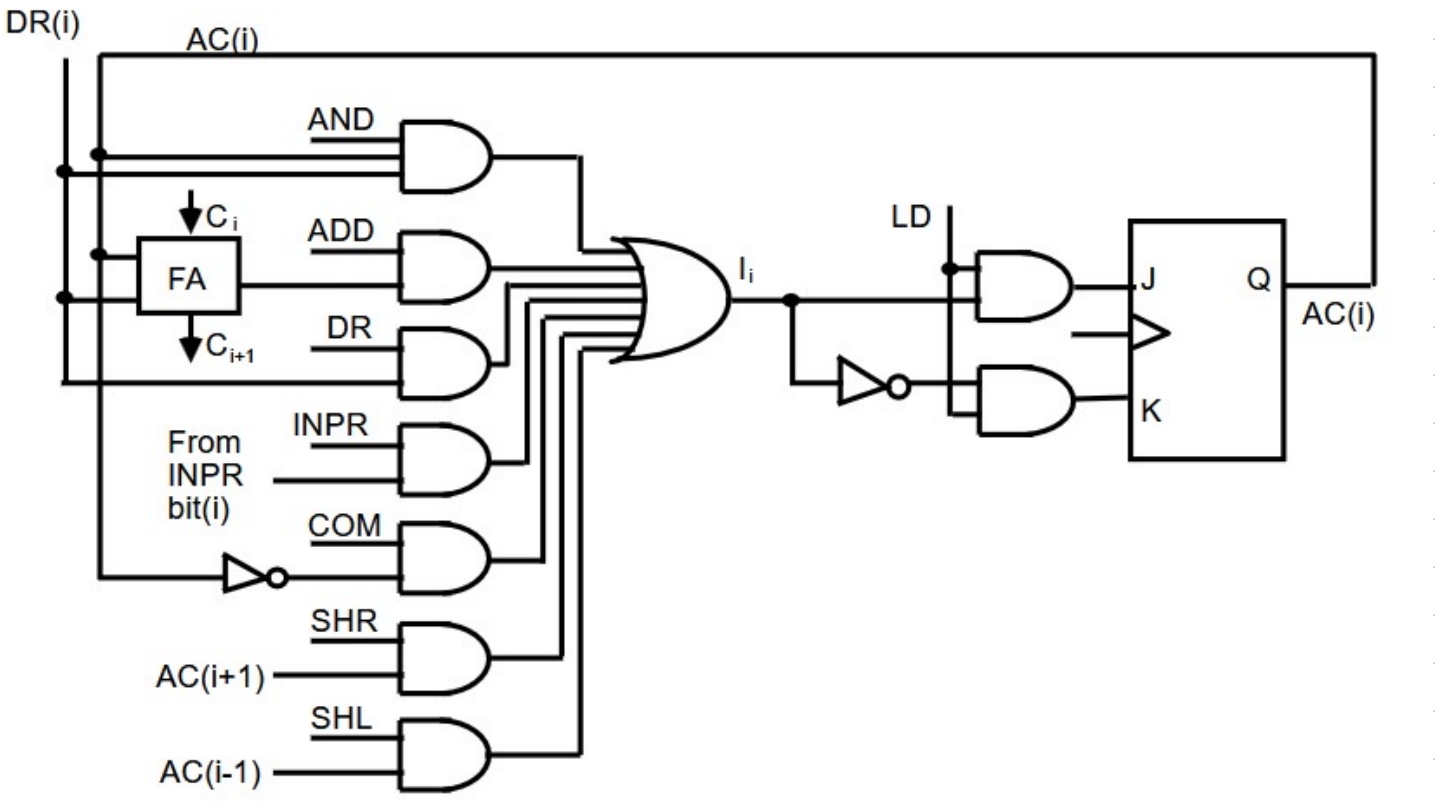
D_0T_5:	$AC \leftarrow AC \wedge DR$	AND with *DR*
D_1T_5:	$AC \leftarrow AC + DR$	Add with *DR*
D_2T_5:	$AC \leftarrow DR$	Transfer from *DR*
pB_{11}:	$AC(0\text{–}7) \leftarrow INPR$	Transfer from *INPR*
rB_9:	$AC \leftarrow \overline{AC}$	Complement
rB_7:	$AC \leftarrow \text{shr } AC, \quad AC(15) \leftarrow E$	Shift right
rB_6:	$AC \leftarrow \text{shl } AC, \quad AC(0) \leftarrow E$	Shift left
rB_{11}:	$AC \leftarrow 0$	Clear
rB_5:	$AC \leftarrow AC + 1$	Increment



control gates



ALU


DR(i)
AC(i)
AND
C_i
ADD
FA
C_{i+1}
DR
INPR
From
INPR
bit(i)
COM
SHR
AC(i+1)
SHL
AC(i-1)
I_i
LD
J
Q
K
AC(i)

Practice Qs:

Q1.

(a) Indirect bit = 1

Op code = 7

Register code = 6

Address = 16

$2^{18} = 262$

(b)

1	7	6	18

(c) Data: 32

Address: 18

Q2. Direct:

1. Read instruction
2. Read operand

Indirect:

1. Read instruction
2. Read effective address
3. Read operand

Q3.

(a) $IR \leftarrow M[AR]$

(b) $PC \leftarrow TR$

(c) $DR \leftarrow AC$

$M[AR] \leftarrow AC$

(d) $AC \leftarrow AC + DR$

Q4.

NOTE: $S_2S_1S_0$ is for those only whose contents needs to be on bus.

(a)

1) 010
2) LD of AR
3) None
4) None

(b)

1) 111
2) LD of IR
3) Read
4) None

(c)

1) 110
2) None
3) Write
4) None

(d)

1) 100
2) LD of AC, LD of DR
3) None
4) Transfer DR to AC

Q5.

(a) $IR \leftarrow M[PC]$

↳ The address line connection is from AR & not from PC

$AR \leftarrow PC$

$IR \leftarrow M[AR]$

(b) $AC \leftarrow AC + TR$

↳ To add into AC you have no connection for TR into ALU.

$DR \leftarrow TR$

$AC \leftarrow AC + DR$

(c) $DR \leftarrow DR + AC$

↳ You need to store add result in AC.

AC ← DR, DR ← AC → swap AC & DR

AC ← AC + DR → store (modify) AC

AC ← DR, DR ← AC → put original value of AC (stored in DR) back to AC & store result in DR.

Q6. I opcode Address

(a) 0001 0000 0010 0100

1 0 2 4

ADD content of M[024] to AC

(b) 1011 0001 0010 0100

B 1 2 4

store content of AC in M[124]

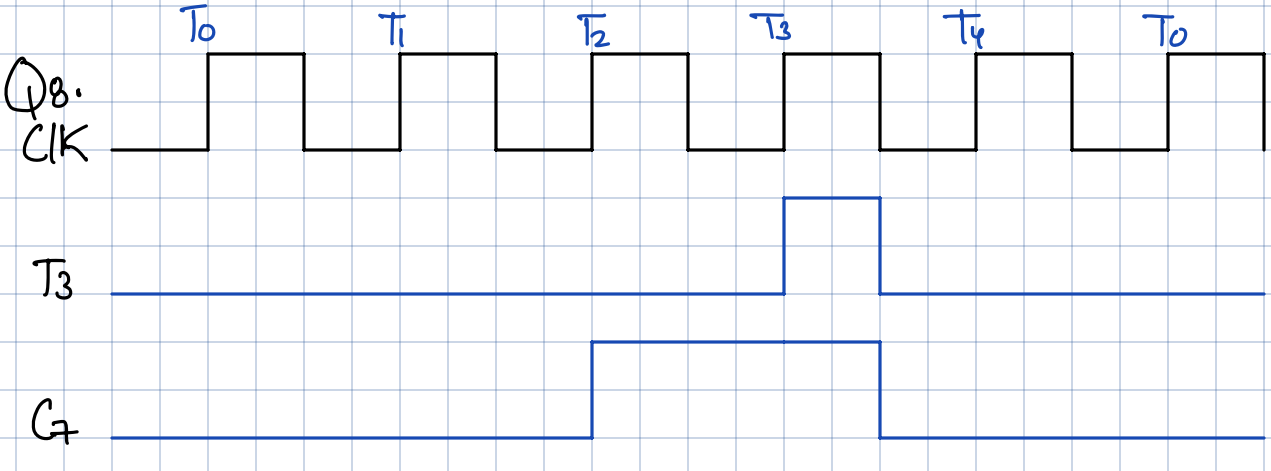
(c) 0111 0000 0010 0000

7 0 2 0

Increment AC

Q7. CLE, CME

Q8.



Q9.

E	AC	PC	AR	IR
1	A937	021	—	—
1	0000	022	800	7800
0	A937	022	400	7400
1	56C8	022	200	7200
0	A937	022	100	7100
1	D49B	022	080	7080
1	526F	022	040	7040
1	A938	022	020	7020
1	A937	022	010	7010
1	A937	022	008	7008
1	A937	022	004	7004
1	A937	022	002	7002
1	A937	022	001	7001

8 4 2 1

1 0 1 0 1001 0011 011

1 1 0 1 0 1001 0011 011

D 4 9 B

Q10.

	PC	AR	DR	AC	IR
Initial	021	—	—	A937	—
	022	083	B8F2	A832	0083
	022	083	B8F2	6229	1083

022	083	B8F2	B8F2	2083
022	083	—	A937	3083
083	083	—	A937	4083
084	084	—	A937	5083
022	083	B8F3	A937	6083

Q11.

	PC	AR	DR	IR	SC
Initial	7FF	—	—	—	0
T_0	7FF	7FF	—	—	1
T_1	800	7FF	—	EA9F	2
T_2	800	A9F	—	EA9F	3
T_3	800	C35	—	EA9F	4
T_4	800	C35	FFFF	EA95	5
T_5	800	C35	0000	EA95	6
T_6	801	C35	0000	EA95	0

7FF = EA9F

A9F = 0C35

C35 = FFFF

Q12.

1 ADD 32E

PC=3AF

AC = 7EC3

(a) Next instruction: 932E

3AF = 932E

ADD 32E to AC

32E = 09AC

9AC = 8B9F

(b) 8B97

+ 7EC3

10A62

1001 0011 0010 1110

ADD

indirect

Address

Fetch operand

E=1

(c) PC = 3AF + 1 = 3B0

AR = 9AC

DR = 8B9F

AC = 0A62

IR = 932E

E = 1

I = 1

SC = 0

Q13.

(a) D_0T_4: DR ← M[AR]

D_0T_5: AC ← AC ⊕ DR, SC ← 0

(b) D_1T_4: DR ← M[AR]

D_1T_5: AC ← AC + DR, DR ← AC

D_1T_6: M[AR] ← AC, AC ← DR, SC ← 0

(c) D_2T_4: DR ← M[AR]

D_2T_5: AC ← DR, DR ← AC → You can't complement DR without loading into AC.

D_2T_5: AC ← $\overline{AC}$

D_2T_5: AC ← AC + 1

D_2T_5: AC ← AC + DR, SC ← 0

(d) D_3T_4: DR ← M[AR]

D_3T_5: M[AR] ← AC, AC ← DR, SC ← 0

(e)

D_4T_4: $DR \leftarrow M[AR]$

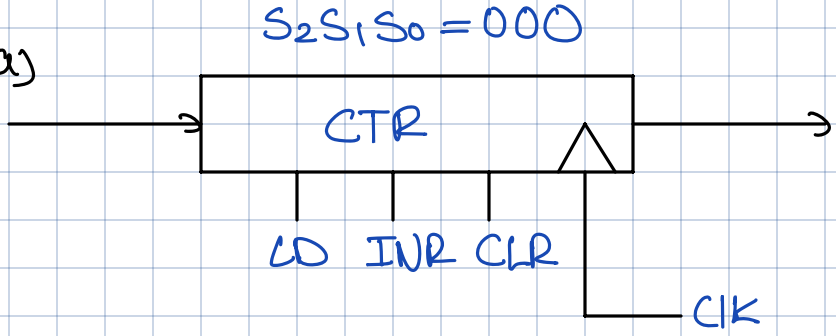
D_4T_5: $TR \leftarrow AC$, $AC \leftarrow AC \oplus DR$

D_4T_5: If $(AC=0)$ then $PC \leftarrow PC+1$, $SC \leftarrow 0$, $AC \leftarrow TR$

(f) D_5T_4: If $(AC \neq 0 \wedge AC(15)=0)$ then $PC \leftarrow AR$, $SC \leftarrow 0$

Q14.

(a)



(b)

LCTR: $CTR \leftarrow M[AR]$

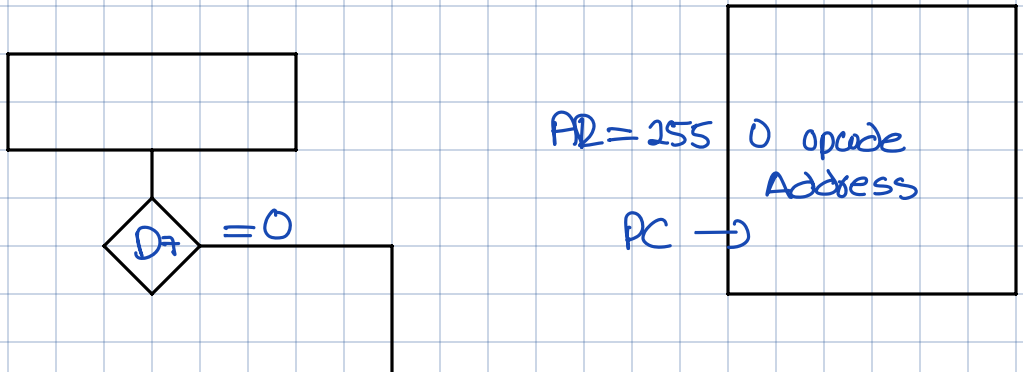
(c) ICSZ:

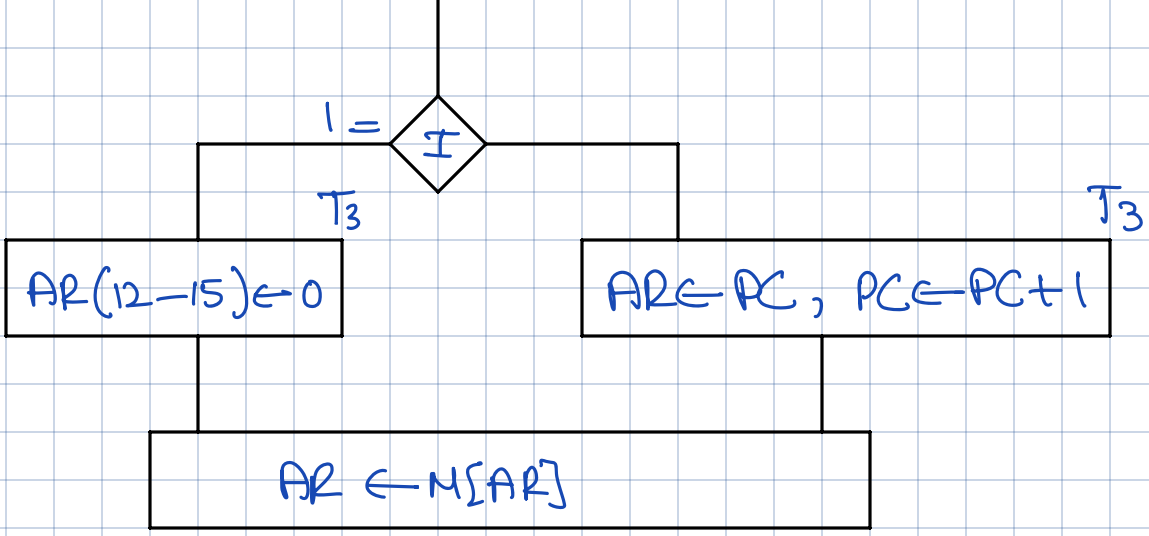
D_6T_3: $CTR \leftarrow CTR+1$,

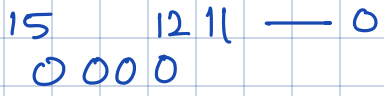
If $(CTR=0)$ $PC \leftarrow PC+1$, $SC \leftarrow 0$

Changing memory-reference instruction into register reference lets us start at T_3 instead of T_6

Q15.

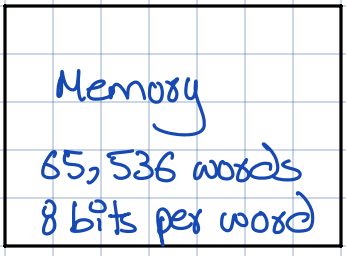


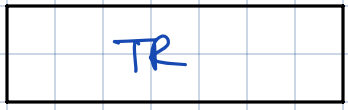


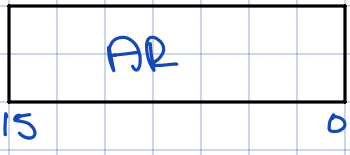


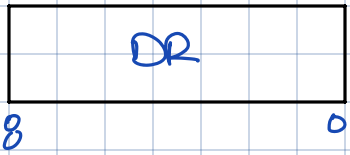
Q16.

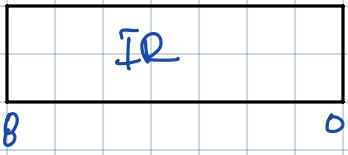
(a)











T_0: AR ← PC, PC ← PC+1

T_1: IR ← M[AR]

T_2: AR ← PC

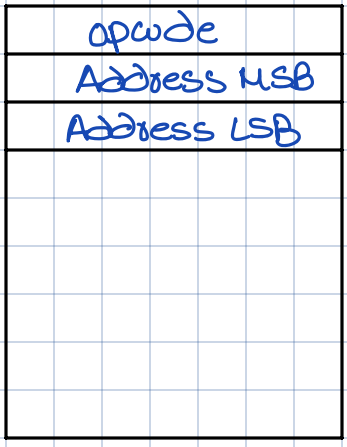
T_3: TR(0–8) ← M[AR], PC ← PC+1

T_4: AR ← PC

T_5: TR(9–15) ← M[AR], PC ← PC+1

T_6: AR ← TR

(b)



WRONG!

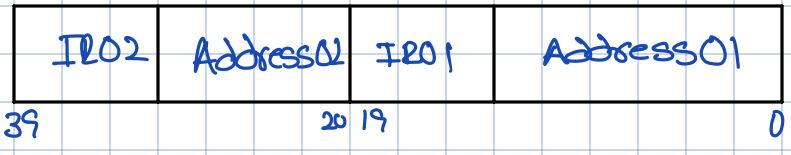
c) T_0: AR ← PC , PC ← PC + 1

T_1: IR ← M[AR], AR ← PC, PC ← PC + 1 → You are selecting both PC & Memory. In one clock cycle, only one register can be selected to put on bus.

T_2: AR(0–7) ← M[AR], TR ← PC, PC ← PC + 1

T_3: AR(8–15) ← M[TR]

Q17.



1. Read 40 bit instruction in memory to IR & Increment PC
2. Decode opcode 01 & execute instruction using address 01
3. Decode opcode 02 & execute instruction using address 02
4. Go to step 1

Q18.

(a) 0 BUN 2300

(b) ION
1 BUN 0